

```

        return None                                # 1 step
    startIndex = 0                                  # 1 step
    endIndex = len(haystack) - 1                    # 1 step

    haystack.sort()                                  # ??? steps

    while start <= end: # ??? steps
        midIndex = (startIndex + endIndex) // 2    # 1 step
        if haystack[midIndex] == needle:           # 1 step
            # Found the needle.
            return midIndex                         # 1 step
        elif needle < haystack[midIndex]:          # 1 step
            # Search the previous half.
            endIndex = midIndex - 1                 # 1 step
        elif needle > haystack[midIndex]:          # 1 step
            # Search the latter half.
            startIndex = midIndex + 1               # 1 step

```

Two of the lines in `binarySearch()` aren't easy to count. The `haystack.sort()` method call's big O order depends on the code inside Python's `sort()` method. This code isn't very easy to find, but you can look up its big O order on the internet to learn that it's $O(n \log n)$. (All general sorting functions are, at best, $O(n \log n)$.) We'll cover the big O order of several common Python functions and methods in "The Big O Order of Common Function Calls" later in this chapter.

The `while` loop isn't as straightforward to analyze as the `for` loops we've seen. We must understand the binary search algorithm to determine how many iterations this loop has. Before the loop, the `startIndex` and `endIndex` cover the entire range of `haystack`, and `midIndex` is set to the midpoint of this range. On each iteration of the `while` loop, one of two things happens. If `haystack[midIndex] == needle`, we know we've found the `needle`, and the function returns the index of the `needle` in `haystack`. If `needle < haystack[midIndex]` or `needle > haystack[midIndex]`, the range covered by `startIndex` and `endIndex` is halved, either by adjusting `startIndex` or adjusting `endIndex`. The number of times we can divide any list of size n in half is $\log_2(n)$. (Unfortunately, this is simply a mathematical fact that you'd be expected to know.) Thus, the `while` loop has a big O order of $O(\log n)$.

But because the $O(n \log n)$ order of the `haystack.sort()` line is higher than $O(\log n)$, we drop the lower $O(\log n)$ order, and the big O order of the